

Conditionals

With conditions, we can make things happen when special criteria are met. Open the aquarium project and try inserting the following into it. The `draw` function:

```
if (fish1X > 500) {  
  fish1X = -40;  
}  
  
if (fish2X < -50) {  
  fish2X = 200;  
}
```

What happens?

Change direction

By making a variable that contains the direction in which the fish swims, we can change the direction when it reaches the sides.

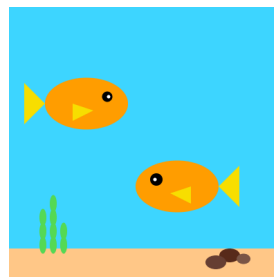
- Define a global variable `fish1XVelocity` and set it to 1
- Change `fish1X = fish1X + 1` to `fish1X = fish1X + fish1XVelocity`
- Remove the previous `fish1X` condition
- Add these conditions:

```
if (fish1X > 400){  
  fish1XVelocity = -1  
}  
if (fish1X < 0){  
  fish1XVelocity = 1  
}
```

Change appearance

Try changing the `drawFish` function to strip the direction with the fish as an argument and use a condition to draw the fins and eyes differently depending on which direction the fish is swimming.

```
function drawFish(x, y, eyeSize, velocity){  
  ... draw body ...  
  
  # Swim to the right  
  if (velocity >= 0){  
    ...draw fins and eyes ...  
  }  
  
  # Swim to the left  
  if (velocity < 0){  
    ...draw fins and eyes ...  
  }  
}
```



Finish the aquarium project

Now, we are almost done with the aquarium project. Add, if necessary, more elements. E.g. bubbles that emerge from the bottom and swim towards the surface.

More exercises about conditions

Open the Green City project and do the following:

- Make the cloud move back to the start and pass by again and again, each time it hits the edge
- Make the car turn over at both ends.

Tasks:

The car must stop when the battery is empty

- Define a global variable `carBattery` and set it to 100
- Decrease it by 0.1 each time `draw` is called
- Display the battery status using the `text(string, x, y)` function. Remember to convert the number to a string using `str()`.
- If the battery is empty (`<= 0`) the car must stop (set velocity to 0)

Add a `keyPressed()` function and make it possible to charge the electric car when you press 'C':

- Add 0.3 to `carBattery` every time 'C' is pressed on the keyboard
- If `carBattery` exceeds 100, set it to 100 so you can't charge more than 100%

Changing wind speed

The variable `frameCount` counts how many times `draw` is run since the program started. Enter this in the draw function i the electric car project:

```
fill(0, 0, 0);  
text(frameCount, 350, 20):  
if (frameCount % 60 == 0){  
  print(frameCount)  
}
```

Since `draw` is executed at *60 frames per second*, will The `print` function will be executed every second.

We can also use that in the electric car project to update the wind speed every second.

- Create a global `windSpeed` variable
- Update `windSpeed` with a new random value every second
- Display the wind speed using `text()`
- Create a global variable `powerplantOn`
- Make it possible to turn the power plant on and off with a press of 'p'
- Draw clouds of smoke over the power plant when it is on
- Create a variable `co2emission` and add a bit as long the power plant is on. Display the value with `text()`

Introduction to Sounds

In this small worksheet, we will add sound files to make sound effects!

1. First check reference <https://p5js.org/reference/p5.MediaElement/play/>
2. Then download the sound files from ABASALON
3. Also check out the sound sketch also uploaded
4. Create a new file and names `sound01`

```
function preload() {
  // Load the sound files before the program starts
  // Load the first sound file (e.g., a music track or sound effect)
  sound = loadSound('sound.mp3');
  // Load the second sound file (e.g., a ball tap sound)
  sound1 = loadSound('balltap.wav');
}

function setup() {
  // Setup the canvas size
  createCanvas(400, 400); // Create a canvas of 400x400 pixels
}

function draw() {
  // Draw the background and instructions
  background(220); // Set the background color to light grey (RGB value: 220)
  // Align text to the center horizontally and vertically
  textAlign(CENTER, CENTER);
  textSize(24); // Set the text size to 24 pixels
  // Display the instructions in the center of the canvas
  text('Press "S" or "A" to play sound', width / 2, height / 2);
}

function keyPressed() {
  // Detect key presses and play the corresponding sound
  if (key === 's' || key === 'S') {
    // Play the first sound when the 'S' key is pressed (case insensitive)
    sound.play();
  }
  if (key === 'a' || key === 'A') {
    // Play the second sound when the 'A' key is pressed (case insensitive)
    sound1.play();
  }
}
```

Descriptions

- **‘preload()’ function:** Loads the sound files into memory before the sketch starts. This ensures that the sounds are ready to be played when triggered.
- **‘setup()’ function:** Sets up the canvas where the sketch will be displayed, specifying its size (400x400 pixels).
- **‘draw()’ function:** Continuously runs in a loop, updating the background color and displaying the instruction text centered on the canvas.
- **‘keyPressed()’ function:** Listens for specific key presses (‘S’ or ‘A’) and plays the corresponding sound when either key is pressed.

Daniel Spikol & Martin Dybdal
ds@di.ku.dk

Introduction to Lists and For Loops

With `for` loops, we can avoid duplicated code and iteratively operate, such as drawing 3 circles on a screen. Start a new project and insert the following code:

```
// Initialize Lists
let xs = [30, 50, 70, 90, 110, 130];
let ys = [30, 50, 70, 90, 110, 130];

function setup() {
  createCanvas(400, 400);
  background(0);
}

function drawCircle(x, y) {
  noStroke();
  bubble_R = random(256);
  bubble_G = random(256);
  bubble_B = random(256);
  fill(bubble_R, bubble_G, bubble_B);
  circle(x, y, 25);
}

function draw() {
  for (var i = 0; i < xs.length; i++) {
    drawCircle(xs[i], ys[i]);
  }
}
```

What happens when you run the code? Explain here how a loop produces the result, and give yourself time to understand the logic.